Punto fijo para filtros FIR

De enteros binarios a formatos Q y cálculos en Arduino

Francisco Carlos Calderon Ph.D

23 de octubre de 2026

Propósito de esta sesión

Al terminar, se debe poder:

- interpretar un entero como un número con punto binario fijo;
- elegir un formato con signo, W bits y F bits fraccionarios;
- calcular el resultado y el tamaño de una suma o multiplicación;
- explicar cada escala usada por un FIR cuantizado.

Idea central: el procesador guarda enteros. Nosotros decidimos qué factor de escala representa cada entero.

Recordatorio: un entero binario

Con cuatro bits sin signo se representan enteros entre 0 y 15:

$$1101_2 = 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 13.$$

Los pesos de los bits son:

posición	3	2	1	0
peso	8	4	2	1

Un entero no lleva punto decimal. El punto fijo asigna algunos bits a pesos menores que uno.

El primer ejemplo de punto fijo

Guardemos el mismo entero 13, pero acordemos dividirlo siempre entre 4:

$$x = \frac{13}{4} = 3{,}25.$$

En binario se puede leer como:

$$1101_2 \quad \longrightarrow \quad 11.01_2 = 3{,}25.$$

El hardware sigue almacenando 1101. El punto no ocupa memoria: es una regla de interpretación compartida por el programa.

Regla general

Si un entero almacenado se llama r y se reservan F bits fraccionarios:

$$\boxed{x = \frac{r}{2^F} = r\,2^{-F}}$$

Ejemplo

$r = 45$ y $F = 4$: $\quad x = 45/16 = 2{,}8125.$

El menor incremento representable se llama LSB:

$$\mathrm{LSB} = 2^{-F}.$$

Más bits fraccionarios dan pasos más pequeños, pero reducen el rango si W permanece constante.

W, F, QF y QI

Usaremos una notación explícita para evitar ambigüedades:

W	número total de bits almacenados
F o QF	bits fraccionarios
QI	bits enteros sin contar el signo
signo	un bit adicional cuando el formato es con signo

Para un formato con signo:

$$W = 1 + QI + QF.$$

Ejemplo: con signo, $W = 8$, $F = 4$. Entonces $QI = 3$ y escribiremos `Q3.4`.

Formato con signo

Los formatos con signo usan complemento a dos. Para $W = 8$ y $F = 4$:

$$\texttt{0010.1101}_2 = 45/16 = 2{,}8125$$

$$\texttt{1110.1100}_2 = (-20)/16 = -1{,}25$$

El bit más a la izquierda tiene peso negativo. En este caso sus pesos son:

$$-2^3,\ 2^2,\ 2^1,\ 2^0,\ 2^{-1},\ 2^{-2},\ 2^{-3},\ 2^{-4}.$$

Rango de un formato

Para un formato sin signo de W bits y F fraccionarios:

$$0 \leq x \leq \frac{2^W - 1}{2^F}.$$

Para un formato con signo:

$$-\frac{2^{W-1}}{2^F} \leq x \leq \frac{2^{W-1} - 1}{2^F}.$$

formato	mínimo	máximo	LSB
sin signo, $W = 8, F = 4$	0	$15{,}9375$	$1/16$
con signo, $W = 8, F = 4$	-8	$7{,}9375$	$1/16$

La notación Q puede cambiar entre libros

Algunos autores cuentan el bit de signo dentro de la parte entera y otros no. Por eso `Q1.15` puede significar cosas distintas según la fuente.

Se recomienda declarar siempre:

$$\boxed{\texttt{signed}, \quad W, \quad F}$$

y mostrar QI y QF sólo como ayuda pedagógica, así se encuentra en varias aplicaciones en línea.

En MATLAB

Un formato con signo, $W = 16$ y $F = 15$ se escribe `numerictype(1,16,15)`.

Codificar y decodificar un número

Ejemplo: formato con signo $W = 8$, $F = 4$.

1. Para guardar $x = 2{,}75$, multiplicar por 2^F:

$$r = 2{,}75 \cdot 16 = 44.$$

2. Almacenar el entero $r = 44$:

$$44 = 00101100_2.$$

3. Para recuperar el valor físico:

$$x = 44/16 = 2{,}75.$$

Si $x\,2^F$ no es entero, hay que escoger un redondeo.

Cuantización y redondeo

En formato $W = 8$, $F = 4$, el valor $0{,}30$ no se representa exactamente:

$$0{,}30 \cdot 16 = 4{,}8.$$

método	entero almacenado	valor representado	error
hacia abajo	4	$4/16 = 0{,}25$	$-0{,}05$
al más cercano	5	$5/16 = 0{,}3125$	$+0{,}0125$
hacia cero	4	$0{,}25$	$-0{,}05$

La elección debe formar parte de la especificación del filtro.

Desbordamiento: saturar

Con signo, $W = 4$, $F = 0$, el rango que cabe en el registro es de -8 a 7. **Suma matemática:**

$7 + 3 = 10$. El resultado correcto es mayor que el máximo representable ($+7$), por lo que no cabe en cuatro bits con signo.

operación	resultado matemático	valor guardado con saturación
$7 + 3$	10	7 (0111)

Regla de saturación

Si el cálculo supera $+7$, se guarda $+7$. Si fuera menor que -8, se guardaría -8. El resultado queda limitado al rango del formato.

Aquí la suma real vale 10, pero el registro conserva el máximo positivo: 0111, que representa $+7$.

Desbordamiento: wrap

Con el mismo formato con signo ($W = 4$, $F = 0$), el hardware conserva sólo cuatro bits del resultado de la suma:

$$\begin{array}{rcll} 0111 & = & +7 & \\ +0011 & = & +3 & \\ \hline 1\,1010 & = & 10 & \text{(resultado de cinco bits)} \\ \,1010 & & & \text{(se descarta el bit que no cabe)} \end{array}$$

Interpretación de los cuatro bits guardados: en complemento a dos, 1010 tiene un bit con peso -8 y otro con peso $+2$:

$$1010_2 = -8 + 2 = -6.$$

Wrap equivale a calcular módulo $2^4 = 16$: $10 \bmod 16 = 10$. La palabra binaria guardada sigue siendo 1010; al leerla como número con signo se obtiene -6. Por eso wrap puede cambiar el signo del resultado.

Tipos usuales en Arduino Uno

El Uno R3 usa el ATmega328P. En el compilador AVR habitual:

tipo	bits	rango con signo
`char` / `int8_t`	8	-128 a 127
`short` / `int16_t`	16	-32768 a 32767
`int`	16	-32768 a 32767
`long` / `int32_t`	32	aproximadamente $\pm 2{,}1 \times 10^9$
`long long` / `int64_t`	64	aproximadamente $\pm 9{,}2 \times 10^{18}$
`float`	32	punto flotante IEEE 754 de precisión simple

El Uno tiene 2 kB de SRAM. Cada bit y cada arreglo importan.

No suponer tamaños de C/C++

El estándar permite que `int` cambie entre plataformas. Para exigir un ancho concreto, usar `<stdint.h>`:

```
#include <stdint.h>
#include <limits.h>

static_assert(sizeof(int16_t) == 2,
              "Se requieren 16 bits");
static_assert(CHAR_BIT == 8,
              "Se requieren bytes de 8 bits");

int16_t muestra;
int32_t acumulador;
```

`int16_t` existe sólo si la plataforma ofrece exactamente 16 bits. `sizeof` devuelve bytes, no bits.

Multiplicación de enteros

Si a usa W_a bits y b usa W_b bits, el producto completo requiere:

$$\boxed{W_p = W_a + W_b}$$

bits como regla segura, tanto para formatos con signo como sin signo.

Ejemplo sin signo

$255 \times 1023 = 260865$. Un operando de 8 bits y otro de 10 bits pueden producir hasta 18 bits de resultado.

Antes de multiplicar en C++, promover los operandos mediante casting ():

```
int32_t producto = (int32_t)a * (int32_t)b;
```

Multiplicación de números de punto fijo

Las escalas también se multiplican:

$$\frac{r_x}{2^{F_x}}\,\frac{r_h}{2^{F_h}} = \frac{r_x r_h}{2^{F_x+F_h}}.$$

Por tanto:

$$\boxed{F_p = F_x + F_h} \qquad \boxed{W_p = W_x + W_h}.$$

Ejemplo: entrada con signo $(W_x, F_x) = (16, 15)$ y coeficiente con signo $(W_h, F_h) = (16, 15)$:

producto: $(W_p, F_p) = (32, 30)$.

Suma de enteros y acumulación

Para sumar dos números de W bits con la misma escala, se necesita un bit adicional en el peor caso:

$$W_{\text{suma}} = W + 1.$$

Para sumar N productos del mismo formato:

$$\boxed{W_{\text{acc}} = W_p + \lceil \log_2(N) \rceil}, \qquad F_{\text{acc}} = F_p.$$

Si las escalas son distintas, primero hay que alinear los puntos binarios. Esa conversión puede introducir redondeo y overflow.

Cadena numérica de un FIR

Para un FIR de 32 taps con entrada y coeficientes $(W, F) = (16, 15)$:

$$\begin{aligned} x[n] &: (16, 15) \\ h[k] &: (16, 15) \\ x[n-k]h[k] &: (32, 30) \\ \sum_{k=0}^{31} x[n-k]h[k] &: (37, 30) \\ y[n] &: (16, 15) \end{aligned}$$

Para pasar del acumulador $(37, 30)$ a la salida $(16, 15)$ se eliminan 15 bits fraccionarios. El redondeo decide cómo eliminarlos y el modo de overflow decide qué ocurre si el resultado no cabe en 16 bits.

Dos significados de normalizar

1. Reescalar un número: cambiar de $F = 30$ a $F = 15$ para guardar una salida. Es una operación de formato y requiere redondeo.

2. Normalizar la ganancia del filtro: escoger los coeficientes para que la ganancia deseada se cumpla. En un pasobajo:

$$H(0) = \sum_k h[k].$$

Si el diseño ideal suma 1, la ganancia DC ideal es uno. Después de cuantizar, hay que revisar:

$$\widehat{H}(0) = \sum_k \widehat{h}[k].$$

Ejemplo: la cuantización puede cambiar la ganancia

Coeficientes ideales de un pasobajo:

$$h = [0{,}2,\ 0{,}6,\ 0{,}2], \qquad \sum h[k] = 1.$$

Con $F_h = 3$, el paso es $1/8$. Al redondear:

$$\widehat{h} = [2/8,\ 5/8,\ 2/8] = [0{,}25,\ 0{,}625,\ 0{,}25].$$

$$\sum \widehat{h}[k] = 1{,}125.$$

La escala numérica sigue siendo correcta, pero el filtro ahora tiene ganancia DC de $1{,}125$. La herramienta debe mostrar ambas ganancias.

Ejercicio manual

Formato de entrada: con signo, $W_x = 8$, $F_x = 4$.

Formato del coeficiente: con signo, $W_h = 8$, $F_h = 4$.

1. Codifique $x = 1{,}5$ y $h = -0{,}75$ como enteros raw.
2. Multiplique los enteros raw.
3. Indique W_p y F_p.
4. Decodifique el producto y compárelo con $1{,}5 \times (-0{,}75)$.

Respuesta esperada: $r_x = 24$, $r_h = -12$, $r_p = -288$, $F_p = 8$ y $-288/256 = -1{,}125$.

Ideas para recordar

- Punto fijo guarda un entero y usa una escala 2^{-F}.
- Declarar `signed`, W y F evita ambigüedades.
- Producto: sumar longitudes de palabra y bits fraccionarios.
- Acumulador FIR: añadir bits de guarda para las sumas.
- Cada conversión de formato necesita una política de redondeo y overflow.
- La ganancia del filtro se revisa después de cuantizar coeficientes.

Fuentes y material para continuar

- Arduino UNO R3: ATmega328P, 16 MHz, 2 kB SRAM y 32 kB Flash.
  `https://docs.arduino.cc/hardware/uno-rev3`
- Arduino, tipos `float`: 32 bits y precisión simple.
  `https://docs.arduino.cc/language-reference/en/variables/data-types/float/`
- C/C++, `<stdint.h>` y los tipos de ancho fijo.
  `https://en.cppreference.com/c/types/integer`
- Laboratorio local: `fir_fixed_lab`, diseño, simulación y exportación compatible con Arduino.